Applied Machine Learning for Business Analytics

Lecture 10: Agent: Memory + Context Engineering

Lecturer: Zhao Rui

Agenda

1. Memory
2. Context Engineering

1. Memory

Memory Landscape

- Short-term Memory *Already covered*
  - Tokens currently in the context window - conversation history, system prompt, tool results
  - Like agent's working desk. Everything visible, but limited space
- Long-term Memory ***Today Focus***
  - Stored outside the context window. Persists across sessions. Selectively retrieved when required.
  - Like agent's personal notebook or knowledge base

Long-term Memory

- Without long-term memory, every conversation session starts from zero
  - The agent can not learn from past interactions
  - Can not remember user preferences
  - Can not build domain expertise over time

> Three types of long-term memory mirror how humans remember.
> — CoALA framework (Cognitive Architectures for Language Agents), adopted by LangChain, Anthropic, and others

Three Types of Long-term Memory

Type	Human Analogy	Agent Examples	Implementation
Semantic Memory	Your professional knowledge base, industry expertise, contact list	User preferences, domain facts, entity knowledge	Structured profiles, key-value stores, vector database
Episodic Memory	War stories, lessons from past projects, "I remember when..."	Past interaction records, what worked or failed in similar situations	Session logs, few-shot examples,
Procedural Memory	How you run meetings, write reports, your standard operating procedures	Learned workflows, behavioral patterns, CLAUDE.md files	System prompts, instruction sets, skills files loaded when relevant

Semantic Memory - What is known

- Facts, concepts, and structured knowledge - the agent's encyclopedia
- Not tied to any specific event - just accumulated know

HUMAN ANALOGY

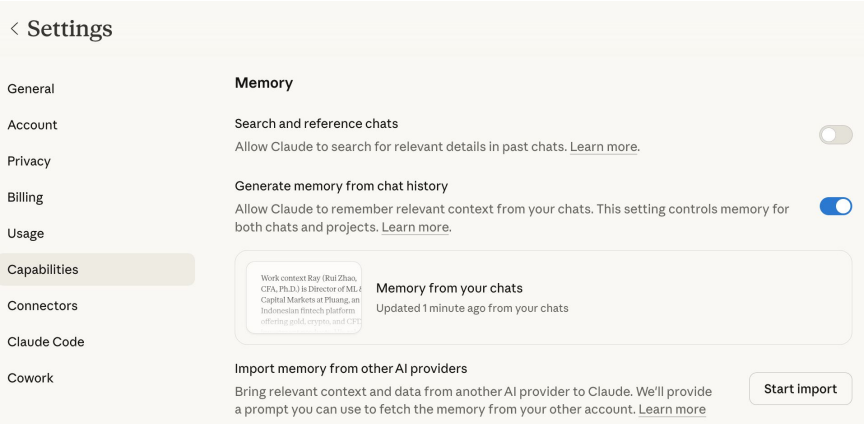
You know company X is using bigquery as database, and your colleague prefers email over Slack.

STORAGE APPROACH

Structured profiles (key-value pairs) or documents in a vector database that can be searched semantically.

⚠️ Key challenge: Reconciling new info with old. If a user moves cities, the agent must *update*, not just append. Over-extraction creates noise; under-extraction causes forgetting.

Semantic Memory - Claude Examples



https://support.claude.com/en/articles/11817273-use-claude-s-chat-search-and-memory-to-build-on-previous-context

Episodic Memory - What happened

- Records of specific past events, interactions and experiences.
- What happened, when, in what context, and what was the outcome. Examples:
  - Last time this user asked about transformers, they wanted numerical examples, not theory
  - Previous attempt to fix the coding issue by modifying the parser failed, fix was in the tokenizer.
- Stored as structured records or as few-shot examples
- It enables **learning from experiences** to avoid same mistakes

⚠️ Key challenge: Retrieval relevance. Not all past episodes apply. Requires good similarity matching beyond surface-level text.

Episodic Memory - Claude Examples

- Claude Code session logs:
  - Store as JSONL format with complete timestamps at ~/.claude/projects/
  - Records every message, tool call and prompts

```
rz@macbookpro projects % pwd
/Users/rz/.claude/projects
rz@macbookpro projects %
```

Claude Code stores full logs of your sessions as newline-delimited JSON in `~/.claude/projects/encoded-directory/*.jsonl` on your machine. I currently have 379MB of these!

Here's an example jsonl file which I extracted from my Deepseek-OCR on NVIDIA Spark project. I have a little vibe-coded tool for converting those into Markdown which produces results like this.

Unfortunately Claude Code has a nasty default behavior of deleting these after 30 days! You can't disable this entirely, but you can at least delay it for 274 years by adding this to your `~/.claude/settings.json` file:

```
{
  "cleanupPeriodDays": 99999
}
```

https://simonwillison.net/2025/Oct/22/claude-code-logs/#:~:text=Claude%20Code%20stores%20full%20logs,currently%20have%20379MB%20of%20these

Procedural Memory - How to do things

- Learned rules, workflow, behavioral patterns, and skills
- Execute without deliberating from scratch each time
- Unlike semantic/episodic which store **data** - procedural stores *behavior*
  - *It can make agents more efficient. Without it, the agent reasons from first principles every time*

HUMAN ANALOGY

You know how to ride a bike, structure a financial report, conduct a code review.

You don't re-derive these processes each time — they're internalized routines.

REAL-WORLD EXAMPLE

Claude Code's Skills (SKILL.md files) are procedural memory — instructions for how to perform specific task types. The agent reads the relevant skill before executing, rather than figuring it out from scratch.

Procedural Memory - Claude Examples

Set up rules

Place markdown files in your project's `.claude/rules/` directory. Each file should cover one topic, with a descriptive filename like `testing.md` or `api-design.md`. All `.md` files are discovered recursively, so you can organize rules into subdirectories like `frontend/` or `backend/`:

```
your-project/
├── .claude/
│   ├── CLAUDE.md           # Main project instructions
│   └── rules/
│       ├── code-style.md   # Code style guidelines
│       ├── testing.md      # Testing conventions
│       └── security.md     # Security requirements
```

https://code.claude.com/docs/en/best-practices

Write an effective CLAUDE.md

Run `/init` to generate a starter CLAUDE.md file based on your current project structure, then refine over time.

CLAUDE.md is a special file that Claude reads at the start of every conversation. Include Bash commands, code style, and workflow rules. This gives Claude persistent context it can't infer from code alone.

The `/init` command analyzes your codebase to detect build systems, test frameworks, and code patterns, giving you a solid foundation to refine.

There's no required format for CLAUDE.md files, but keep it short and human-readable. For example:

CLAUDE.md

```
# Code style
- Use ES modules (import/export) syntax, not CommonJS (require)
- Destructure imports when possible (eg. import { foo } from 'bar')

# Workflow
- Be sure to typecheck when you're done making a series of code changes
- Prefer running single tests, and not the whole test suite, for performance
```

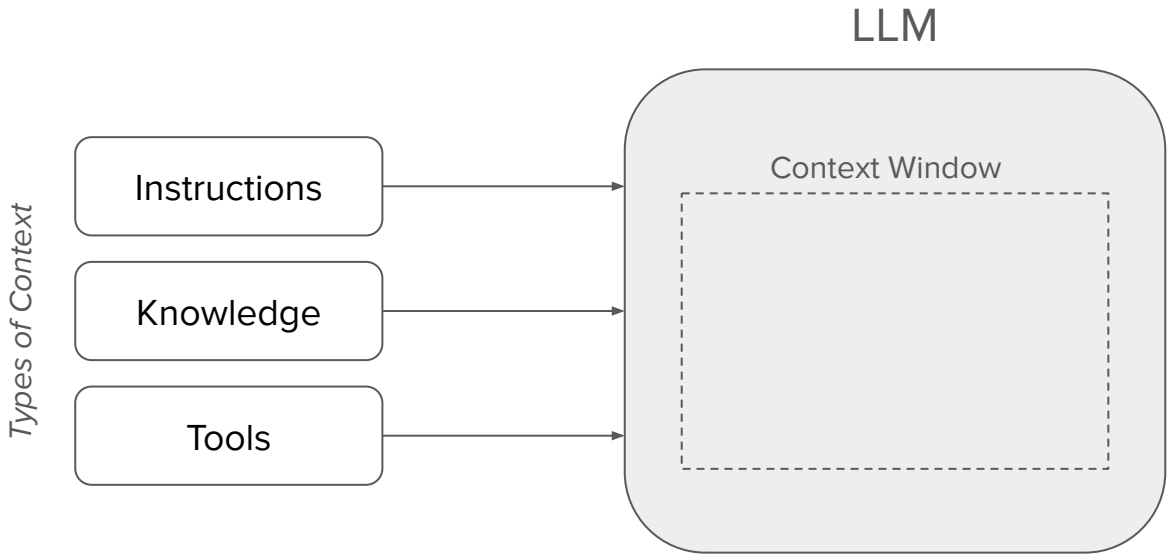
Memory & The Human Career

Memory Type	Human Career	Agent Equivalent
Short-term	What's on your desk right now	Within the session — conversation history, system prompt, tool results
Semantic	Professional knowledge base, contact list, industry expertise	Facts & preferences store — user profiles, domain knowledge
Episodic	War stories, lessons from past projects, "I remember when..."	Past interaction records — what worked, what failed, and why
Procedural	Standard operating procedures, how you run meetings, how you write reports	Skills & workflows — CLAUDE.md files, instruction sets, SKILL.md files

2. Context Engineering

What is Context

- Context, i.e., the model input concatenating
  - **Instructions** - Prompts, memories, few-shots examples, tool descriptions, etc
  - **Knowledge** - facts, memories, etc
  - **Tools** - feedback from tool calls



From Prompt Engineering to Context Engineering

- Prompt Engineering
  - Find the right words and phrases to make the model behave as desired. Work for single-turn, one-shot tasks
- Context Engineering
  - Managing what information the model has access to at each step across the agent lifecycle

⚠️ As models get smarter, prompt engineering matters less while context engineering matters more - even the smartest model fails without the right information at the right time

Prompt vs Context

Intelligence is not the bottleneck - context is. By Antropic

	Prompt Engineering	Context Engineering
Focus	What words to use in the prompt	What information to put in the context window
Scope	Single LLM call	Across the entire agent lifecycle
Time horizon	Static (write once)	Dynamic (curate at each step)
Primary challenge	Phrasing and instruction design	Information selection, compression, and flow
Where value comes from	Clever instruction writing	Thoughtful information architecture
As models improve	Matters less — smarter models follow imperfect instructions	Matters more — even smart models fail without the right information

Context Engineering

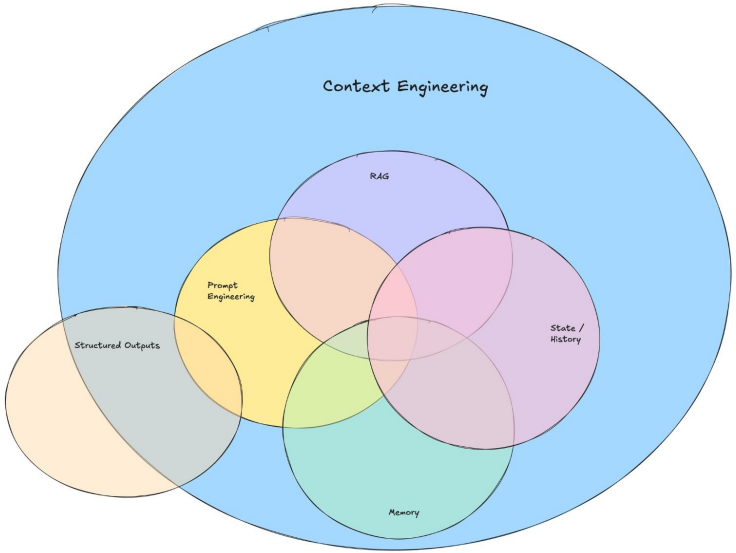
Karpathy's Framing

LLM = CPU. Context window = RAM

Context engineering is like OS memory management - deciding what to load into RAM, what to keep on disk, and when to swap.

Anthropic's Definition

The set of strategies for curating and maintaining the optimal set of tokens during LLM inference



Context Window

The max tokens that LLM model can handle at one time. Once exceeded, the oldest info is lost

The smallest unit of processing.

Model	Context Window (Token size)
Gemini 2.5 Pro	1M
Claude Opus 4.6	200k
GPT5	400K

Why We need to manage context

- Context is limited for most of LLM models
  - Massive windows are the exception. 50k tokens is common for some small models
- Garbage in, Garbage out
  - Dump redundant, or noisy data confuses the model’s focus, directly leading to hallucinations and vague outputs
- Cost
  - API calls are billed per token. Unoptimized input scales infra costs exponentially without improving accuracy.

Why Context Engineering Matters Now

- Model is stronger and stronger
  - Those recent LLMs are incredibly capable. For most of tasks, model limitations are not the cause of the poor performances
- The rise of Agent
  - Due to reasoning, tool execution and multi-turn characteristics behind agent, it can generate a super long result log. Without management, those logs can rapidly occupy the context window, degrading performance over time.

Context engineering is the delicate art and science of filling the context window with just the right information for the next step (in an agent's trajectory)

When Context is not Managed Well

Things that go wrong when context is poorly managed:

Failure Mode	What Happens	Analogy
Context Poisoning	A hallucination or error from an earlier step enters the context and corrupts all subsequent reasoning	A wrong assumption in meeting notes that nobody corrects — every decision built on it is flawed
Context Distraction	Too much irrelevant information overwhelms attention. Models can fail to use information that's present simply because there's too much else around it	Trying to find one email in a completely unorganized inbox
Context Confusion	Overlapping or contradictory information leads to inconsistent outputs	Giving someone two conflicting memos about the same project
Context Clash	History + tool results + memory + instructions exceeds the window limit. Something must be dropped — but what?	Running out of desk space; shoving important papers off the edge

Four Context Engineering Strategies

Four operations on context - unified framework from Anthropic, LangChain and Agent community.

Write

Proactively save information to external storage so it survives context limits and can be retrieved later.

taking notes during a call

Select

Retrieve only the relevant subset of available information when needed. Not everything stored should be loaded.

Pulling the right report before a call

Compress

Reduce token count of information in context while preserving essential meaning. Same info, less space.

Summary a 50-page report into 2 pages

Isolate

Separate context into difference scopes so each LLM call sees only what it needs, without pollution from other tasks.

Different teams with their own contexts

These strategies compose and reinforce each other - not used in isolation

Write

Proactively save information to external storage (outside the context windows) so it survives context limits

- Scratchpads: within agent session
  - “Note taking” via a scratchpad is one approach to persist information while an agent is performing a task
  - It can be implemented via
    - A tool call that writes to a file
    - A runtime state object that persists during the session

> research process. The LeadResearcher begins by thinking through the approach and saving its plan to Memory to persist the context, since if the context window exceeds 200,000 tokens it will be truncated and it is important to retain the plan. It then creates specialized Subagents

https://www.anthropic.com/engineering/multi-agent-research-system

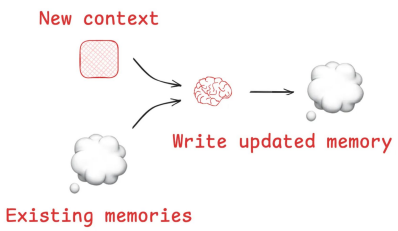
When to use: Any tasks that might exceed the context windows, or any information that needs to persist across sessions or agent handsoff.

Write

Proactively save information to external storage (outside the context windows) so it survives context limits

- Scratchpads: within agent session
- Memories: across agent sessions
  - Based on user-agent interactions, all of popular agents have mechanisms to auto-generate long-term memories

Dimension	ChatGPT	Cursor	Claude
Storage Location	OpenAI server	Local filesystem	Antropic server
Write trigger	Auto-detected or user-explicit	Manual / agent post session update	Auto-detected or user-explicit



Select

Retrieve the useful and available information into the context windows to help agent perform tasks

- Scratchpad:
  - We can select context based on how we implemented scratchpad
  - It can be loaded fully or partially (like what parts of state to expose to an agnet)

```python
class AgentState(MessagesState):
    """
    Main state for the full multi-agent research system.

    Extends MessagesState with additional fields for research coordination.
    Note: Some fields are duplicated across different state classes for proper
    state management between subgraphs and the main workflow.
    """

    # Research brief generated from user conversation history
    research_brief: Optional[str]
    # Messages exchanged with the supervisor agent for coordination
    supervisor_messages: Annotated[Sequence[BaseMessage], add_messages]
    # Raw unprocessed research notes collected during the research phase
    raw_notes: Annotated[list[str], operator.add] = []
    # Processed and structured notes ready for report generation
    notes: Annotated[list[str], operator.add] = []
    # Final formatted research report
    final_report: str
```

State object in Langchain
https://github.com/langchain-ai/deep_research_from_scratch/blob/main/notebooks/1_scoping.ipynb

Select

Retrieve the useful and available information into the context windows to help agent perform tasks

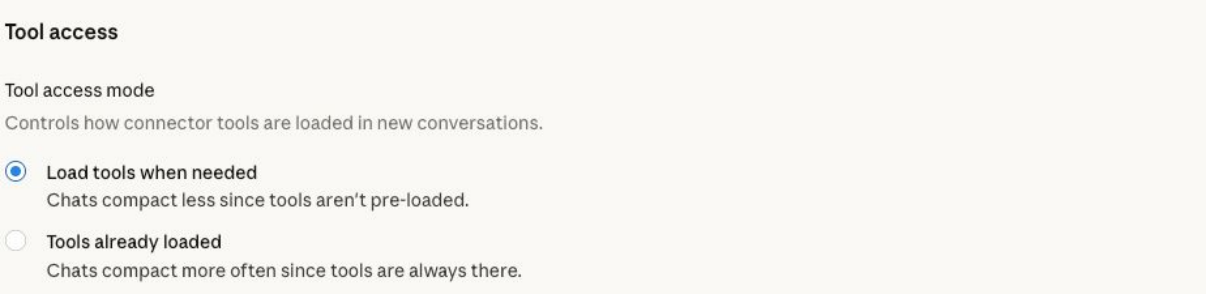
- Scratchpad
- Memories
  - After memories are saved, we can select memories relevant to the task that agents are performing. It is useful because
    - Semantic->facts as task-relevant context
    - Episodic->few-shot examples
    - Procedural->instructions
  - The challenge here is how to ensure relevant memories are selected
    - Procedural -> save in files -> load fully into the context everytime
    - Semantic/Episodic->memory store -> load dynamically (becoming a retrieval question)

which make context selection very challenging

Select

Retrieve the useful and available information into the context windows to help agent perform tasks

- Scratchpad
- Memories
- Tools
  - Tools already loaded: All tool descriptions are presented in every context windows -> overload
  - Load tools when needed: Tool descriptions are only loaded into context windows when it is needed -> Another retrieval problem



Compress

Reduce token count while preserving essential meaning. Same info, less space.

- Context Summarization:
  - Conversation compaction:
    - Claude code's `/compact` command summaries earlier turns while retaining critical details
  - Tool result summarization:
    - Instead of keeping a full 10,000-token API response in context, summarize it to the 200 tokens actually relevant
  - Progressive summarization:
    - As conversation grows, older turns get progressively more summarized - recent turns stay detailed, older turns become bullet points.
    - Claude Code will auto-compact if 95% of tokens are used.

When to use: long conversations, large tool outputs, or any time context is growing faster than new information is being added.

How to overcome lossy compression?

Compress

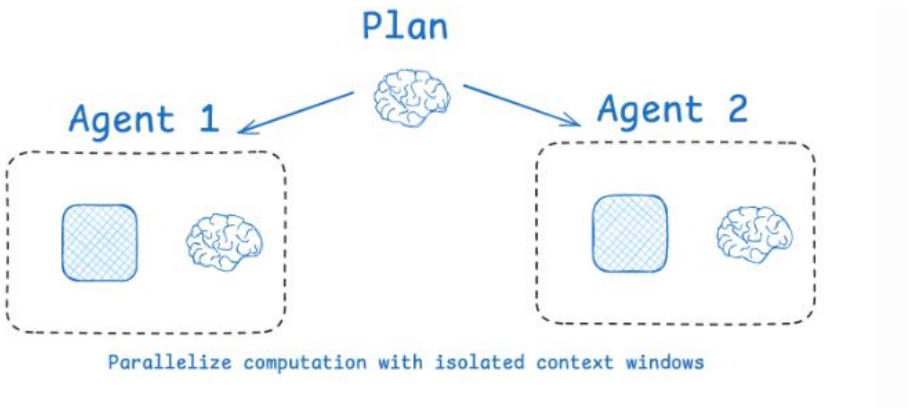
Reduce token count while preserving essential meaning. Same info, less space.

- ~~Context Summarization:~~
- Context Trimming
  - Removing messages based on some conditions when the context length exceed

Isolate

Separate context into scopes so each LLM call sees only what it needs

- Multi-agent architecture:
  - Multi-agent research system assign sub-agent each operating with their own context window. Each agent handle sub-tasks with its own:
    - A specific set of tools, instructions and its own context windows

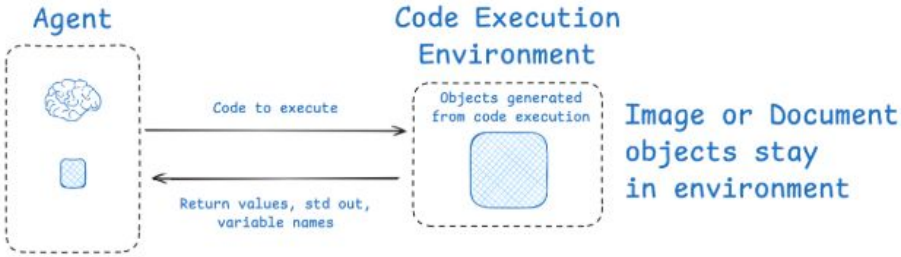


When to use: complex tasks with independent subtasks, when cross-contamination of context would cause confusion or errors. And it is closely connected to orchestration patterns.

Isolate

Separate context into scopes so each LLM call sees only what it needs

- Multi-agent architecture
- Context Isolation with Environment:
  - LLM -> JSON tool call -> Tool executes -> Full result -> Back to LLM context
  - Code Agent: LLM writes the actual code
    - Outputs code that contains the deserted tool calls, the code runs in a sandbox, selected context from the output is passed back to LLM




```
# Call multiple tools and combine results in one step
r1 = search("Memory management for Agents")
r2 = search("Context engineering for Agents")
combined = r1["results"] + r2["results"]
top3 = sorted(combined, key=lambda x: x["relevance"])[:3]
```

LLM-> programming its own tool use
Sandbox -> run the code safely, return only the final variables back

Wrap it up quickly

Reduce token count while preserving essential meaning. Same info, less space.

- Write context
  - Saving it outside the context window
- Selecting context
  - Pulling it into the context window
- Compressing context
  - Retain the tokens required to perform a task
- Isolating context
  - Split the context up

The Complete Picture

- Memory determines what the agent stores
- Tool determines what the agent can see and act
- Thinking patterns determines what the agent can reason
- Orchestration determines what the agents can collaborate
- Context engineering determines what reaches the model at each step
  - As a **cross-cutting discipline** - ensures information flows correctly between all three levels.
  - Like the OS that manages RAM for all running process

Together, they turn a stateless and single-turn LLM into a continuously improving and context-aware agent.

Memory vs Context Engineering

- Memory - *What the agent knows or has learned*
  - Persistent state that survives across sessions or tasks
- Context engineering - *What the agent sees right now*
  - Managing the content inside the active context window for a specific inference call.

	Memory	**Write Context**
Purpose	Persist knowledge for future use	Offload current context to storage
Scope	Cross-session, long-lived	Within-task or cross-step
Who reads it	Future agent usages	The same agent later in the same task
Example	User profile, learned preference	Scratchpad, Intermediate reasoning, tool results

Illusion of Control

With LLM, we can never be certain of anything.

- Context engineering can definitely make agents more effective and increase their success probability.
- However, execution still depends on how well the LLM interprets them!
  - Those features like “ensure it generates JSON format” or “no hallucination” is not there for LLM applications or agents

We need to think in probabilities for Agent Development and Usage. Human in the loop at the right level is the key

Next Class: LLM Eval